

In our architecture, *JournalService* acts as a consumer whereas *AddService* acts as a producer, along with the *FindService* and *SearchService*.

AddService as Kafka producer

In order to produce messages to the Kafka broker, the *AddService* requires the Kafka library. Spring Boot comes with a ready-to-use library for Kafka. Add the following snippet as one of the dependencies in the *pom.xml* of the *AddService*.

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

For ease of use, add the following declaration in the *build* section of the *pom.xml*. It names the resulting package as *ums-add-service.jar* at the end of the build process.

```
<finalName>ums-add-service</finalName>
```

Now, let's move on to the actual code. Spring Boot makes Kafka integration much simpler by offering a template named *KafkaTemplate*. Let us inject the template into the *UserController* of the *AddService*.

```
@Autowired
private KafkaTemplate<String, UserRecord> template;
```

This statement says that we intend to produce a Kafka message with a *String* as the key and *UserRecord* as the value.

Then, the following code does the job of actually publishing the message to Kafka broker:

```
template.send("com.glarimy.ums.user.add", record);
```

The first argument is the topic name and the second argument is the message value. We choose *com.glarimy.ums.uesr.add* as the topic name. This topic acts as the virtual channel between the producer (*AddService*) and the consumer (*JournalService*) to notify the event of user additions.

Here is the complete code:

```
package com.glarimy.ums.app;
import java.util.Set;
import javax.validation.ConstraintViolation;
import javax.validation.Validator;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.
Autowired;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.
RestController;
import com.glarimy.ums.domain.Name;
import com.glarimy.ums.domain.PhoneNumber;
import com.glarimy.ums.domain.User;
import com.glarimy.ums.domain.UserRepository;
@RestController
public class UserController {
    Logger logger = LoggerFactory.getLogger(UserController.
class);
    @Autowired
    private UserRepository repo;
    @Autowired
    private Validator validator;
    @Autowired
    private KafkaTemplate<String, UserRecord> template;
    @PostMapping("/user")
    public ResponseEntity<UserRecord> add(@RequestBody NewUser
newUser) {
        logger.debug("Received new user");
        Name name = new Name(newUser.name);
        PhoneNumber phoneNumber = new PhoneNumber(newUser.phone);
        logger.debug("Building User domain object");
        User user = new User(name, phoneNumber);
        logger.debug("Validating User domain object");
        Set<ConstraintViolation<User>> violations = validator.
validate(user);
        ResponseEntity<UserRecord> response;
        if (!violations.isEmpty()) {
            violations.forEach(v -> logger.debug(v.getMessage()));
            response = new ResponseEntity<>(HttpStatus.BAD_
REQUEST);
            logger.debug("Response: " + response);
        } else {
            logger.debug("Saving the User domain object");
            repo.save(user);
            logger.debug("Building the response");
            UserRecord record = new UserRecord(user.getName().
getValue(), user.getPhone().getValue(), user.getSince());
            template.send("com.glarimy.ums.user.add", record);
            response = new ResponseEntity<>(record, HttpStatus.OK);
            logger.debug("Response: " + response);
        }
        logger.debug("returning the response");
        return response;
    }
}
```